# Implementing Multi-Turn Conversation Memory

// Phase 4: State management for stateless LLMs
// Transitioning from search engine to conversational assistant

## Phase 3 (No Memory)

**User**

What is the minimum average for that?

**System**

I cannot determine what "that" refers to in the provided documents.

## Phase 4 (With Memory)

campus transfer

**User**

What is the minimum average for that?

**System** ✓

The minimum average for a campus transfer is 80%.

SECTION B-B

MEMORY WINDOW

## The Reality

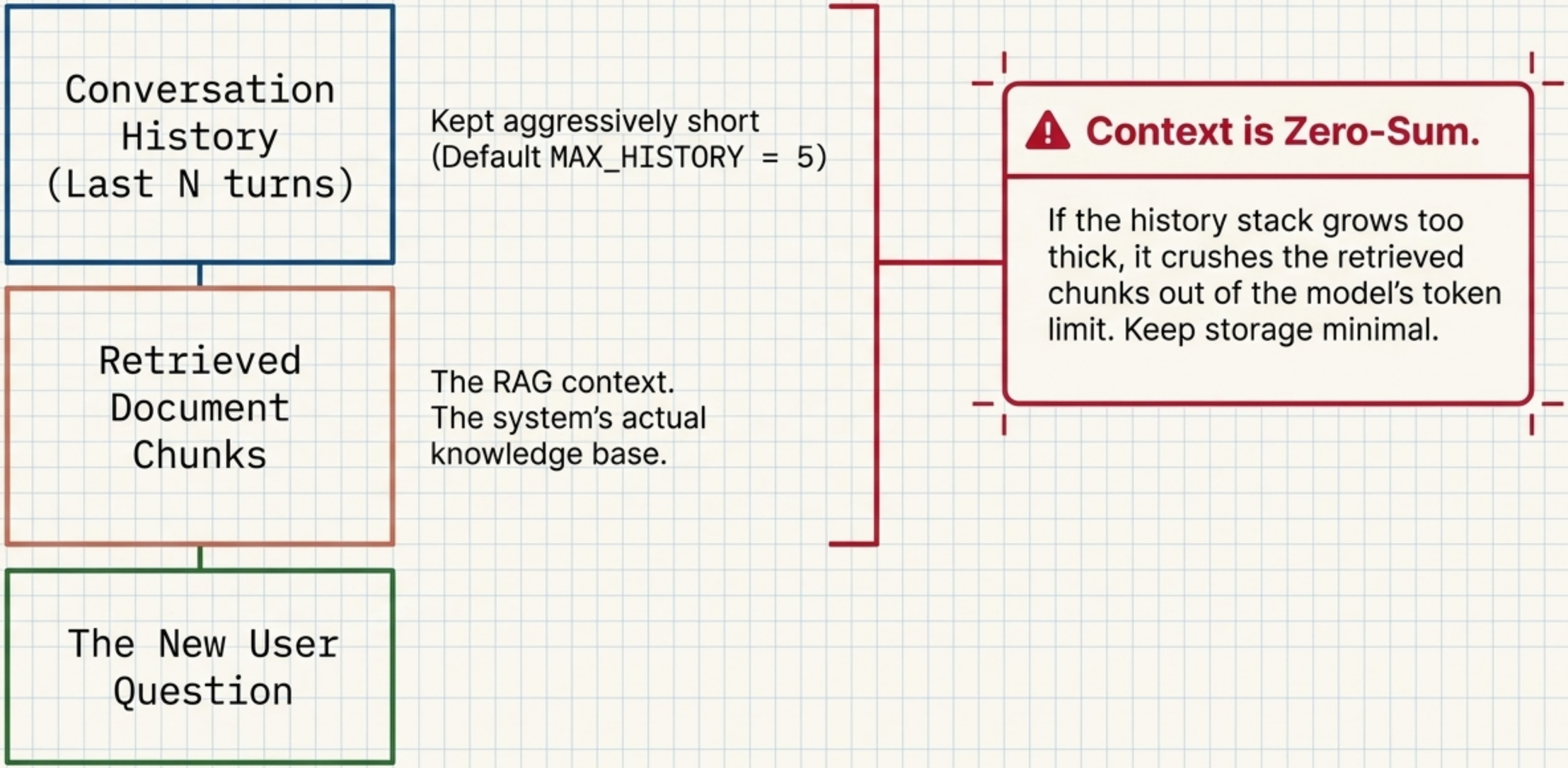
Every API call starts completely fresh. The model remembers neither your last question nor its last answer.

## The Illusion

Memory is an injected artificial construct, created through precise string manipulation.


## The Mechanism

The Context Window. We store history externally and force-feed it to the model with every new prompt.


Conversation History (Last N turns)
Retrieved Document Chunks
The New User Question
Kept aggressively short (Default MAX_HISTORY = 5)
The RAG context. The system's actual knowledge base.
Context is Zero-Sum.
If the history stack grows too thick, it crushes the retrieved chunks out of the model's token limit. Keep storage minimal.

```
{
  'session_id': 'uuid-generated-on-load',
  'timestamp': 'last-activity-time',
  'turns': [
    {
      {'q': 'The exact user question',
       'a': 'The system response'}
    ]
  ]
}
```

## Strictly Excluded

- [x] Retrieved Document Chunks
- [x] Model Metadata & Turn Timestamps

## Sliding Context Window

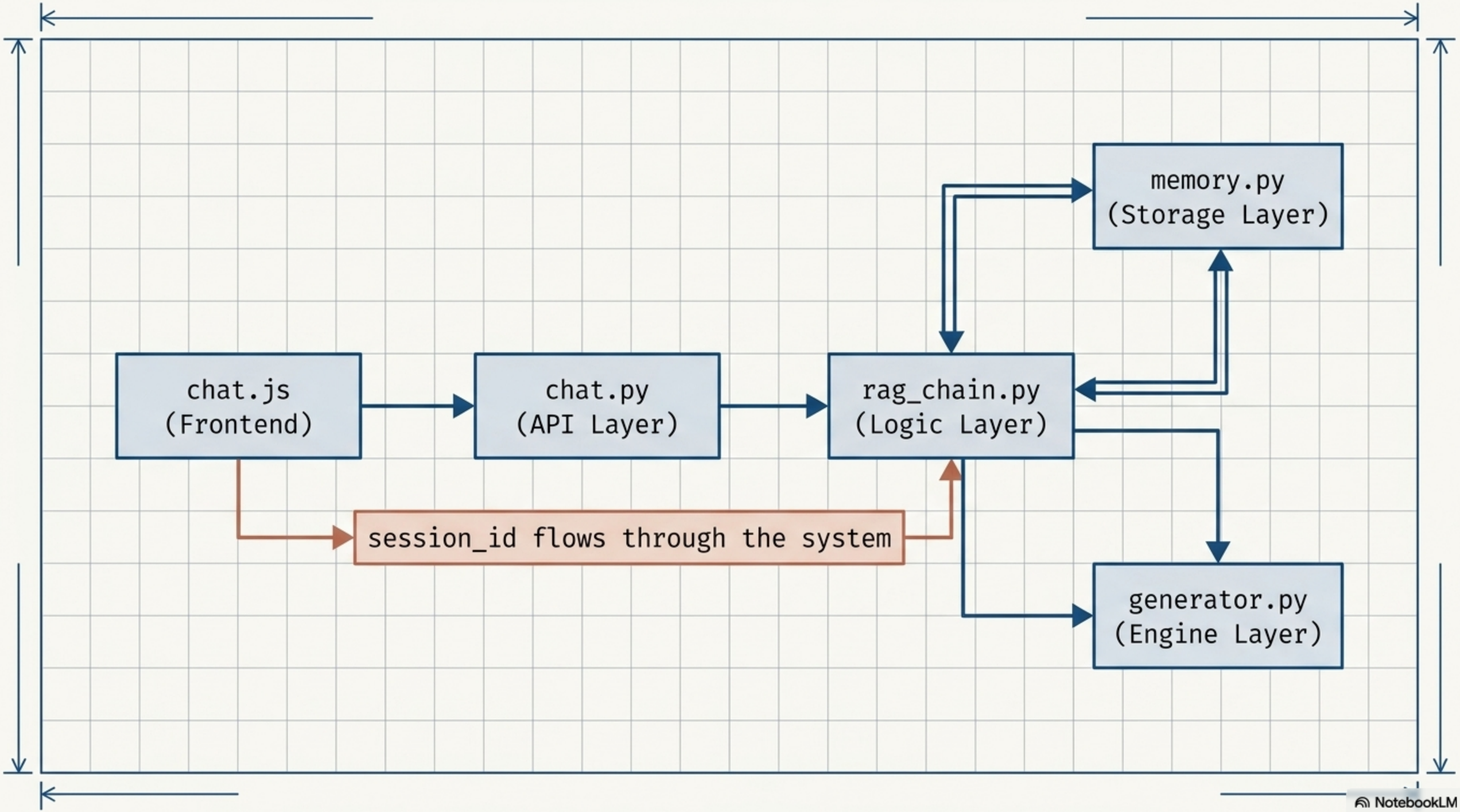
User: What is the minimum average for a campus transfer? ->
System: 80%.

User: Can I do it more than once?

User: What about specializations?

User: Who should I speak to about it? ->
System: The campus director.

None of these follow-up questions mention transfers explicitly. The system resolves them all via the sliding context window.


memory.py
(Storage Layer)
chat.js
(Frontend)
chat.py
(API Layer)
rag_chain.py
(Logic Layer)
session_id flows through the system
generator.py
(Engine Layer)

## The Memory Service (memory.py)

`add_turn(session_id, question, answer)`
Appends turn to history. Triggers expiry checks.

`get_history(session_id)`
Retrieves the last MAX_HISTORY turns for prompt injection.

`clear_session(session_id)`
Deletes session. Triggered by frontend clear action.

Implementation: A Python Dictionary. Everything lives in RAM. No databases, no external services.


Lazy Eviction: Expired sessions are deleted automatically during normal usage. No background threads. No scheduled cron jobs.

| Dimension | In-Memory Dictionary (memory.py) | Production External (Redis) |
| --- | --- | --- |
| Speed | Instant (RAM) | Sub-millisecond (Network) |
| Persistence | **Lost** on server restart | **Survives** restarts |
| Scaling | **Single-server** only | **Shared** across multi-server |
| Complexity | **Zero** setup | **Requires** infrastructure config |

The architecture supports upgrading. Swapping the dictionary for Redis is a 1-file change.

# The Three Rules of LLM Memory

## 1. Memory is an Illusion

Models are inherently stateless. We force them to read their history as injected context.

## 2. Context is a Zero-Sum Game

Conversation history directly competes with RAG retrieval. Keep history short to preserve knowledge bandwidth.

## 3. Complexity Must Be Earned

Start simple. In-memory dictionary storage is perfect for development. Upgrade to Redis only when multi-server scaling demands it.